

MVP V1.1 - MICROSOFT FOUNDRY GPT-SOL EDITION

BidSight

AI Vendor Quotation Evaluation System Software Requirements, System Design and Implementation Specification

Document status	Implementation-ready and aligned with the current repository
Primary stack	Next.js + FastAPI + SQLite + PyMuPDF + Microsoft Foundry
AI deployment	Configurable Foundry deployment; default name gpt-sol
Updated	11 August 2026

Decision-support principle

Microsoft Foundry gpt-sol interprets quotation language and explains verified results. Deterministic Python code performs compliance checks, arithmetic, scoring, and ranking. The final purchasing decision remains with an authorised user.

Prepared for the BidSight portfolio project by Badar Butt

1. Executive summary

BidSight is an AI-assisted procurement decision-support application. A user creates an evaluation, defines purchasing requirements, uploads up to three vendor quotation PDFs, reviews structured quotation facts, and receives a deterministic comparison plus an evidence-based recommendation.

Current implementation choice

BidSight calls a Microsoft Foundry model deployment directly through the OpenAI-compatible /openai/v1/ endpoint. The configured deployment name is gpt-sol by default. It is not a chatbot and it is not a Foundry hosted agent or multi-agent system.

1.1 MVP success statement

- Create a procurement evaluation with quantity, budget, currency, delivery target, and notes.
- Define mandatory and preferred purchasing requirements.
- Upload up to three digitally generated, text-readable PDF quotations.
- Extract fields through Foundry gpt-sol and correct them through a human review gate.
- Run reproducible Python compliance, price, weighted scoring, and ranking.
- Generate a Foundry explanation from verified structured evidence only.

1.2 Repository truth

Layer	Implemented responsibility
Next.js frontend	Evaluation workflow, uploads, editable review, comparison, recommendation display
FastAPI backend	REST API, validation, orchestration, persistence, error handling
PyMuPDF	Page-by-page text extraction from text-readable PDFs
Foundry gpt-sol	Structured quotation extraction and recommendation wording
Python scoring	Mandatory gates, price formula, component scores, final ranking
SQLite	Evaluations, requirements, quotations, results, recommendation state

2. Scope, users, and workflow

2.1 In scope

Capability	MVP requirement
Evaluations	Create, list, reopen, and update a procurement evaluation.
Requirements	Store mandatory/preferred rules with values, units, and operators.
Quotation upload	Accept PDF only and enforce a maximum of three quotations.
AI extraction	Map extracted text to a strict Pydantic quotation schema.
Human verification	Allow edits and require review before scoring.
Scoring	Run deterministic Python compliance and weighted scoring.
Recommendation	Use Foundry only after verified Python results exist.

2.2 Explicitly out of scope

- Authentication, multi-tenant accounts, purchase order creation, and approval chains.
- OCR for scanned quotations, RAG, vector databases, and vendor communication automation.
- Foundry hosted agents, multi-agent orchestration, background workers, or message queues.
- PostgreSQL, cloud object storage, and production deployment automation.

2.3 Primary user

A procurement officer, IT administrator, education purchasing team, or small-business owner comparing competing vendor quotations.

2.4 End-to-end workflow

DRAFT -> REQUIREMENTS_READY -> QUOTATIONS_UPLOADED
 -> REVIEW_REQUIRED -> READY_FOR_SCORING
 -> SCORED -> RECOMMENDATION_READY

Step	System action	Authority
1	Create evaluation and requirements	User + FastAPI
2	Validate and store quotation PDF	FastAPI + PDF service
3	Extract page text	PyMuPDF
4	Structure quotation facts	Foundry gpt-sol
5	Review and correct extracted values	Human user
6	Calculate compliance and scores	Python scoring service
7	Explain recommendation	Foundry gpt-sol
8	Make purchasing decision	Authorised user

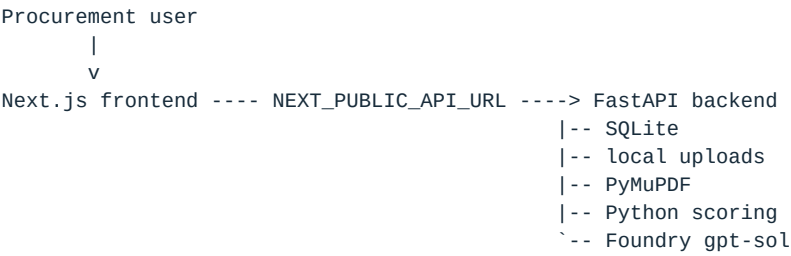
3. Functional requirements

ID	Requirement	Priority
FR-001	Create evaluations with title, category, quantity, budget, currency, delivery days, and notes.	Must
FR-002	Create and list mandatory or preferred requirements with optional unit and operator.	Must
FR-003	Accept PDF uploads only; reject invalid, corrupted, empty, oversized, or fourth quotations.	Must
FR-004	Extract page-numbered text from text-readable PDFs through PyMuPDF.	Must
FR-005	Call Foundry gpt-sol through an isolated backend service and require structured output.	Must
FR-006	Preserve missing or ambiguous values as null and list missing information.	Must
FR-007	Allow users to edit extracted commercial and technical values before confirmation.	Must
FR-008	Block scoring until all uploaded quotations are reviewed.	Must
FR-009	Perform mandatory compliance and weighted scoring only in Python.	Must
FR-010	Keep non-compliant vendors visible with failure explanations.	Must
FR-011	Generate recommendations only after verified structured results are available.	Must
FR-012	Reject a Foundry recommendation for an unknown or ineligible vendor.	Must
FR-013	Expose recoverable processing errors and preserve the uploaded PDF for retry.	Must
FR-014	Provide dashboard, evaluations, vendors, settings, and workflow navigation.	Should

4. Non-functional requirements

Area	Requirement
Reliability	A Foundry or PDF failure must not corrupt an evaluation or delete the stored quotation.
Integrity	Use Decimal-compatible values for stored money and never delegate arithmetic to the model.
Security	Read secrets from untracked environment files and do not expose provider tracebacks.
Maintainability	Keep PDF, Foundry, and scoring logic in separate backend services.
Testability	Scoring and API tests must run without a live Foundry call.
Accessibility	Use labelled forms, keyboard-friendly controls, meaningful statuses, and adequate contrast.
Portability	Use uv for Python and pnpm for Next.js package management.

5. System architecture



5.1 Component boundaries

Module	Boundary
frontend/lib/api.ts	All browser-to-backend HTTP calls and response typing.
pdf_service.py	PDF validation, safe storage, path controls, and page text extraction.
foundry_service.py	Foundry client creation, prompts, structured response validation, provider errors.
scoring_service.py	Compliance, price baseline, weighted score, eligibility, and rank.
routers	Workflow orchestration, status transitions, persistence, HTTP errors.

5.2 Processing sequence

- The frontend posts the evaluation and requirements to FastAPI.
- FastAPI validates and stores each PDF under a server-generated safe filename.
- PyMuPDF extracts text and inserts explicit page markers.
- Foundry receives untrusted quotation text under a system prompt and returns typed fields.
- The user confirms or edits every quotation.
- Python creates compliance checks, scores, ranks, risks, and missing-information lists.
- Foundry receives only verified structured evidence and Python results for final wording.

6. Microsoft Foundry gpt-sol design

6.1 Supported integration

The backend uses the OpenAI Python SDK against the Foundry resource OpenAI v1 endpoint. FOUNDRY_MODEL_DEPLOYMENT is sent as the model parameter and therefore must equal the deployment name shown in Microsoft Foundry. The default repository value is gpt-sol.

```
FOUNDRY_ENDPOINT=https://YOUR-RESOURCE-NAME.services.ai.azure.com
FOUNDRY_API_KEY=your_local_secret
FOUNDRY_MODEL_DEPLOYMENT=gpt-sol
FOUNDRY_REQUEST_TIMEOUT_SECONDS=90
```

Endpoint rule

Use the resource endpoint, not a Playground page URL and not a project-management URL. The backend accepts the resource root or the same URL ending in /openai/v1/.

6.2 Responsibility A - quotation extraction

Input	Output	Guardrail
Page-marked PDF text	FoundryQuotationExtraction	No invention; null for unclear values
Untrusted document content	Numbers, terms, specifications, missing fields	Ignore instructions embedded in the quotation

6.3 Responsibility B - recommendation generation

Supplied evidence	Required output
Evaluation and requirements	Recommended vendor
Reviewed quotation fields	Concise reasoning and strengths
Python scores, status, ranks, and checks	Risks, missing information, cheaper-vendor tradeoff

Hard boundary

Foundry must not calculate or change compliance, price score, weighted score, eligibility, or rank. The API rejects a model-selected vendor that is absent or ineligible.

7. Structured output contracts

7.1 FoundryQuotationExtraction

```

vendor_name: string | null
product_name: string | null
product_model: string | null
quantity: integer | null
unit_price: number | null
subtotal: number | null
tax: number | null
total_price: number | null
currency: string | null
delivery_days: integer | null
warranty_months: integer | null
payment_terms: string | null
support_details: string | null
quotation_validity_days: integer | null
specifications: object
missing_information: string[]
source_pages: object

```

7.2 FoundryRecommendation

```

recommended_vendor: string
concise_reasoning: string
strengths: string[]
risks: string[]
missing_information: string[]
tradeoff_explanation: string

```

7.3 Validation and failure behavior

- Pydantic validates structured output before any extracted values are persisted.
- An empty, refused, or malformed response becomes a controlled provider error.
- The API returns a human-readable 502 or 503 response without exposing credentials or traces.
- The uploaded quotation remains stored so the user can correct configuration and retry.

8. Deterministic scoring

Criterion	Weight	Authority
Price	35%	Python formula using lowest eligible positive price
Technical compliance	30%	Python requirement checks
Delivery	15%	Python rule against required days
Warranty	10%	Python rule against required months
Payment terms	5%	Explicit Python rule
Support	5%	Explicit Python rule

```
price_score = lowest_eligible_price / vendor_price * 100
```

```

overall = price*0.35 + technical*0.30 + delivery*0.15
          + warranty*0.10 + payment*0.05 + support*0.05

```

9. Compliance and ranking rules

- A clearly failed mandatory requirement produces NON_COMPLIANT.
- An unavailable mandatory value produces UNKNOWN and MISSING_INFORMATION, not assumed compliance.
- Only compliant or partially compliant vendors with a positive price set the price baseline.
- Non-compliant vendors stay visible and are ranked after eligible vendors.
- Zero or missing price is ineligible for the price formula and safely produces a zero price score.

9.1 Required sample behavior

Vendor	Evidence	Expected status
TechCore Solutions	16 GB RAM, 10-day delivery, 36-month warranty	COMPLIANT
Digital Systems	18-day delivery against 14-day maximum	NON_COMPLIANT
Future Computers	8 GB RAM and 12-month warranty	NON_COMPLIANT

10. Data model

Entity	Purpose and key fields
Evaluation	Title, category, quantity, budget, currency, delivery target, status, recommendation
Requirement	Evaluation link, name, value, unit, type, operator
Quotation	Evaluation link, safe file metadata, extracted/reviewed values, processing state
VendorResult	Quotation link, compliance, component scores, overall score, risks, missing fields

10.1 Relationships

```

Evaluation 1 --- * Requirement
Evaluation 1 --- * Quotation
Quotation 1 --- 0..1 VendorResult

```


11. REST API

Method	Path	Purpose
POST	/api/evaluations	Create evaluation and optional requirements
GET	/api/evaluations	List evaluations
GET/PATCH	/api/evaluations/{id}	Read or update evaluation
POST/GET	/api/evaluations/{id}/requirements	Create or list requirements
POST/GET	/api/evaluations/{id}/quotations	Upload or list quotations
DELETE	/api/quotations/{id}	Remove quotation and safe stored file
POST	/api/quotations/{id}/process	Extract PDF text and call Foundry
PATCH	/api/quotations/{id}/extraction	Confirm corrected extraction
POST	/api/evaluations/{id}/evaluate	Run deterministic scoring
GET	/api/evaluations/{id}/comparison	Read vendor comparison
POST	/api/evaluations/{id}/recommendation	Generate Foundry explanation

12. Frontend and UX requirements

- Enterprise application shell with dashboard, evaluations, vendors, settings, search, and profile.
- Guided new-evaluation form with reusable requirements builder.
- Three-file upload cap, PDF validation feedback, processing state, removal, and retry.
- Editable extraction review with visible missing and uncertain values.
- Comparison table with price, compliance, delivery, warranty, scores, status, and failure evidence.
- Recommendation panel with strengths, risks, missing information, and tradeoff explanation.
- Responsive empty, loading, error, and success states; no chatbot presentation.

13. Configuration and local setup

13.1 Backend environment

```
DATABASE_URL=sqlite:///./bidsight.db
FOUNDRY_ENDPOINT=https://YOUR-RESOURCE-NAME.services.ai.azure.com
FOUNDRY_API_KEY=
FOUNDRY_MODEL_DEPLOYMENT=gpt-sol
FOUNDRY_REQUEST_TIMEOUT_SECONDS=90
UPLOAD_DIR=uploads
FRONTEND_URL=http://localhost:3000
MAX_UPLOAD_SIZE_MB=10
```

13.2 Frontend environment

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

13.3 Commands

```
# Backend
cd backend
uv sync --group dev
uv run uvicorn app.main:app --reload

# Frontend (second terminal)
cd frontend
pnpm install
pnpm dev
```

Dependency note

The provider dependency changed to the OpenAI Python SDK. Regenerate backend/uv.lock with uv sync after this migration.
Do not treat the pre-migration lock as current.

14. Security and error controls

- Never commit .env, API keys, runtime PDFs, SQLite databases, or frontend local env files.
- Use safe generated filenames and validated paths under backend/uploads.
- Treat quotation text and structured evidence as untrusted data in model prompts.
- Do not return provider exceptions, request payloads, keys, or full traces to the browser.
- Rotate a key immediately if it appears in source, documentation, screenshots, or Git history.

15. Test strategy and acceptance criteria

15.1 Automated checks

- Scoring: compliance, RAM, delivery, warranty, missing values, prices, weights, and ranking.
- PDF: MIME/extension, empty file, upload size, safe filename, and path traversal.
- Foundry service: missing config, endpoint normalization, empty/malformed output, valid parsed output.
- API flow: create, upload, mocked extraction, review, three-vendor scoring, recommendation, comparison.
- Sample data: PDF facts, arithmetic, status, and TechCore recommendation consistency.

15.2 Exact commands

```
cd backend
uv run pytest
```

```
cd ../frontend
pnpm typecheck
pnpm lint
pnpm build
```

15.3 Acceptance criteria

- All routes are reachable with matching frontend request/response shapes.
- The three sample quotations remain reviewable and non-compliant vendors remain visible.
- TechCore emerges because of evidence and deterministic eligibility, not vendor-name logic.
- A missing or malformed Foundry response is recoverable and does not delete the quotation.
- README and all three docs match the actual Foundry-based repository.

16. MVP boundaries and future scope

BidSight remains a focused portfolio MVP. Future work should follow demonstrated need rather than precede a reliable local workflow.

Future capability	When to consider
OCR	When scanned/image quotations become a real input requirement
Authentication	When evaluations require user ownership and access control
PostgreSQL	When concurrent hosted persistence is required
Cloud object storage	When uploaded PDFs must survive beyond local disk
Formal report export	When procurement teams require signed or downloadable reports

17. Definition of done

- Frontend and backend start from documented commands after dependencies are installed.
- Foundry environment values are configured locally and a live extraction can be retried safely.
- Database and upload directory initialise at application startup.
- Sample PDFs and expected-results.json remain consistent.
- Automated backend tests and frontend checks pass without a live provider call.
- No previous-provider, Foundry hosted-agent, or multi-agent code remains in the MVP path.

18. Engineering references

Microsoft Foundry Models endpoints: <https://learn.microsoft.com/azure/foundry/foundry-models/concepts/endpoints>

Structured outputs: <https://learn.microsoft.com/azure/foundry/openai/how-to/structured-outputs>

FastAPI: <https://fastapi.tiangolo.com/>

Next.js: <https://nextjs.org/docs>

uv: <https://docs.astral.sh/uv/>

Engineering handoff

Build and demonstrate one reliable path: requirements, three quotations, Foundry extraction, human review, deterministic Python scoring, and a grounded Foundry recommendation.